MobilityDB Hiking Workshop

How to use MobilityDB to analyze hiking data

Alice Lombard and Raphaël Dubuget

COLLABORATORS

	TITLE : MobilityDB Hiking Workshop		
ACTION	*NAME*	*DATE*	*SIGNATURE*
WRITTEN BY	Alice Lombard and Raphaël Dubuget	July 11, 2024	

REVISION HISTORY

NUMBER	DATE	DESCRIPTION	NAME

Contents

1 Analyzing the data **1**

1.1 Loading GPX data 1

1.2 Transforming the data 2

2 Visualizing the data **4**

2.1 Introduction 4

2.2 Connecting to the database 5

2.3 Visualizing the data 5

List of Figures

2.1 New PostGIS Connection 5
2.2 OpenStreetMap layer 6
2.3 Full temporal visualization 6
2.4 Visualizing the trip 7

List of Tables

2.1 Download the move.zip file 4

Abstract

This document is an example of how you can use MobilityDB in order to analyse data. It is directly inspired by the MobilityDB Workshop already existing. We will throughout this document present different use cases of MobilityDB, which is an extension on PostGIS and PostgreSQL.

This workshop was made by two students from INSA Rennes during an internship in June, July and August 2024.

We will use data from OpenStreetMap. The traces we are going to use in this example are rando_morbi1.gpx and rando_morbi2.gpx. GPX is a common file format for storing GPS data. It is used by many GPS devices and applications to store track logs and waypoints. In our case, a person did two hikes on the 26th of June 2024, in the Morbihan, and saved the data in two GPX files. Then, this person exported the data to OpenStreetMap and we can use his trace as we want. We will use these GPX files in this workshop. Please download the files rando_morbi1.gpx and rando_morbi2.gpx from the OpenStreetMap website and put in a folder you will remember.

Now, we are ready to start the workshop. While this workshop illustrates the usage of MobilityDB functions, it does not explain them in detail. If you need help concerning the functions of MobilityDB, please refer to the documentation.

Chapter 1

Analyzing the data

1.1 Loading GPX data

GPX, or GPS Exchange Format, is an XML data format for GPS data. Location data (and optionally elevation, time, and other information) is stored in tags and can be interchanged between GPS devices and software. Conceptually, a GPX file contains tracks, which are a record of where a moving object has been, and routes, which are suggestions about where it might go in the future. Furthermore, both tracks and routes and composed by points. The following is a truncated (for brevity) example GPX file.

```
<?xml version='1.0' encoding='UTF-8' standalone='yes' ?>
<gpx version="1.1"
xmlns="http://www.topografix.com/GPX/1/1"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.topografix.com/GPX/1/1
http://www.topografix.com/GPX/1/1/gpx.xsd"
creator="Example creator">
<metadata>
  <name>Dec 14, 2014 4:32:04 PM</name>
  <author>Example creator</author>
  <link href="https://..." />
  <time>2014-12-14T14:32:04.650Z</time>
</metadata>
<trk>
  <name>Dec 14, 2014 4:32:04 PM</name>
  <trkseg>
    <trkpt lat="30.16398" lon="31.467701">
      <ele>76</ele>
      <time>2014-12-14T14:32:10.339Z</time>
    </trkpt>
    <trkpt lat="30.16394" lon="31.467333">
      <ele>73</ele>
      <time>2014-12-14T14:32:16.00Z</time>
    </trkpt>
    <trkpt lat="30.16408" lon="31.467218">
      <ele>74</ele>
      <time>2014-12-14T14:32:19.00Z</time>
    </trkpt>
    [...]
  </trkseg>
  <trkseg>
    [...]
  </trkseg>
  [...]
</trk>
```

```
<trk>
  [...]
</trk>
[...]
<gpx>
```

To load the GPX data into MobilityDB, we fare going to use ogr2ogr, a command line tool that converts data between different formats. First, you need to create a database. Let's name it "hiking". Here is how you can do it:

```
sudo -u user psql -d postgres -c "CREATE DATABASE hiking;"
```

Of course you need to replace user by your username. Then, you need to load the MobilityDB extension into the database. Here is how you can do it:

```
CREATE EXTENSION mobilitydb CASCADE;
```

We are now ready to import the data. Go to the folder where you have put the GPX files and run the following command:

```
ogr2ogr -append -f PostgreSQL PG:dbname=hiking 11390305.gpx
```

You need to replace 11390305 by the name of the file you want to import. Please run this command with both files. Now, you should have the data in the database.

1.2 Transforming the data

Let's have a look at the table "tracks" and "track_points" which were created by ogr2ogr.

```
SELECT * FROM track_points;
SELECT * FROM tracks;
```

You can see that the table "track_points" contains the points of the track, and the table "tracks" contains the entire tracks. We are going to work with the table "track_points". First, we see that the two tracks are in the same table. We are going to put an ID for each track so that we can differentiate them. Here is how you can do it:

```
ALTER TABLE track_points ADD COLUMN track_number int;

WITH NumberedTracks AS (
SELECT *, LAG(track_seg_point_id) OVER (ORDER BY ogc_fid) AS prev_value
FROM track_points
),
TrackIdentifiers AS (
SELECT ogc_fid, SUM(CASE WHEN track_seg_point_id < prev_value THEN 1 ELSE 0 END) OVER ( ↩
    ORDER BY ogc_fid) + 1 AS track_id
FROM NumberedTracks
)
UPDATE track_points tp
SET track_number = ti.track_id
FROM TrackIdentifiers ti
WHERE tp.ogc_fid = ti.ogc_fid;
```

What we are doing here is that we are adding a column "track_number" to the table "track_points". Then, we are creating "NumberedTracks" which contains the previous value of the "track_seg_point_id" column. Then, we are creating "TrackIdentifiers" which contains the track_id, depending on the previous track_seg_point_id: if it is smaller then we sum 0 but if it is higher, saying we start a new sequence in track_seq_point_id, we sum 1. Finally, we are updating the table "track_points" with the track_id.

We have to transform these geopgraphic points into a MobilityDB trajectory. Here is how you can do it:

```
DROP TABLE IF EXISTS trips_mdb;
CREATE TABLE trips_mdb (
id int,
date date,
trip tgeompoint,
trajectory geometry,
PRIMARY KEY (id)
);

INSERT INTO trips_mdb(id, date, trip)
SELECT track_number, date(time), tgeompointSeq(array_agg(tgeompoint(
wkb_geometry, time) ORDER BY time))
FROM track_points
GROUP BY track_number, date;

UPDATE trips_mdb
SET trajectory = trajectory(trip);
```

What we are doing here is that we are creating a table "trips_mdb" with an id, a date, a trip and a trajectory. The trip is a tgeompoint, which is a MobilityDB type. We are inserting into this table the track_number, the date and the tgeompointSeq of the array of the tgeompoint. Finally, we are updating the table with the trajectory of the trip.

We are done with the data loading. In conclusion, we have loaded the GPX data into MobilityDB, and we have created a table "trips_mdb" which contains the trajectories of the trips. We are now ready to visualize the data.

Chapter 2

Visualizing the data

2.1 Introduction

Now that we have cleaned the data and made it ready to be used, we will visualize the data in order to have a better understanding of it. To do so, we are going to use QGIS, a free and open-source geographic information system. You can download it here. If you're interested, know that we are going to follow this tutorial: MobilityDB: Hands on Tutorial on Managing and Visualizing Geospatial Trajectories in SQL. We will use the plugin Move to visualize the data. Here is the repo where you can find it. Please click on the move.zip file, and download it. Take care to do it like in this following image, otherwise it could not be downloaded correctly. Also, put it in a folder you will remember. Now, open QGIS and click on Plugins → Manage and Install Plugins. Click

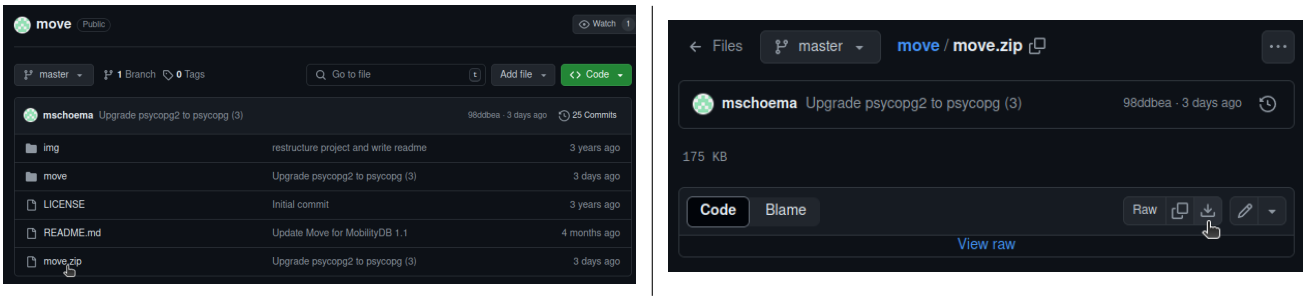

Table 2.1: Download the move.zip file

on the install from zip button and select the move.zip file you just downloaded. Then, you can use the plugin to visualize the data. A MobilityDB logo will appear on the up-left corner of the screen.

You may have an error saying:

```
Couldn't load plugin 'move' due to an error when calling its classFactory() method
    ModuleNotFoundError: No module named 'psycopg'
```

To fix this, you need to install the psycopg module. You can do it by running the following command in a terminal:

```
pip install psycopg
```

If you've had to install the psycopg module, you will need to uninstall the incorrect move plugin that you just installed (Plugins → Manage and Install Plugins → Installed → Move → Uninstall Plugin) and then reinstall it. You may also need to restart QGIS to make the plugin work.

2.2 Connecting to the database

You can now create a new blank project in QGIS. Then in the browser on the left of the screen, you should find a 'PostGIS' line. Right click on it and then 'New Connection...'. You will see this window:

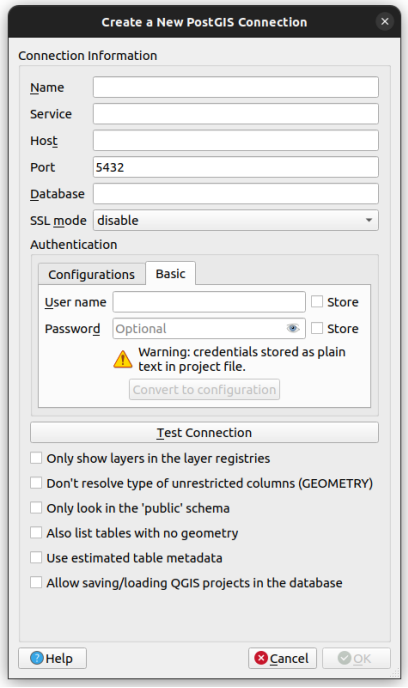

Figure 2.1: New PostGIS Connection

Here is what you need to fill out:

- Name: choose a name for this connection
- Host: if the database is hosted on your machine, then 'localhost'. If not, you should be able to fgure it out by yourself.
- Port: again, if self hosted then keep '5432', if not you should know.
- Database: here write the name of the database in which you have loaded the data. In our case, it was "hiking".
- Authentication: go to the 'Basic' tab and fill out the user name and password for the account that has access to the database. Make sure to check the 'Store' boxes next to User name and Password.

You can now click OK and find your connection, database and the trips_mdb table in the tree below the PostGIS line in the QGIS Browser.

2.3 Visualizing the data

Now that you have connected to the database, you can visualize the data. First, let's put a map layer. In the browser window (on the left of the screen), click on 'XYZ Tiles' and then right click on 'OpenStreetMap' and select 'Add Layer to Project'. You will see an OpenStreetMap layer in the inferior left part of the screen. You can zoom in the part you are studyin. In our case, we are focusing on a part of France, in Britanny, called Morbihan.

Then, let's add the data we have loaded in the database. First, let's draw the trajectory. Click on PostGIS → connection_name → public → trips_mdb → Fields → trajectory. You will see the trajectory in the map. Now, we want to visualize the moving point of the trip. To do so, you need to click on the 'Move' logo on the up-left side of the screen. Select your database and the table you want to visualize. Then, you can write a query and execute it. We are going to put in this query:

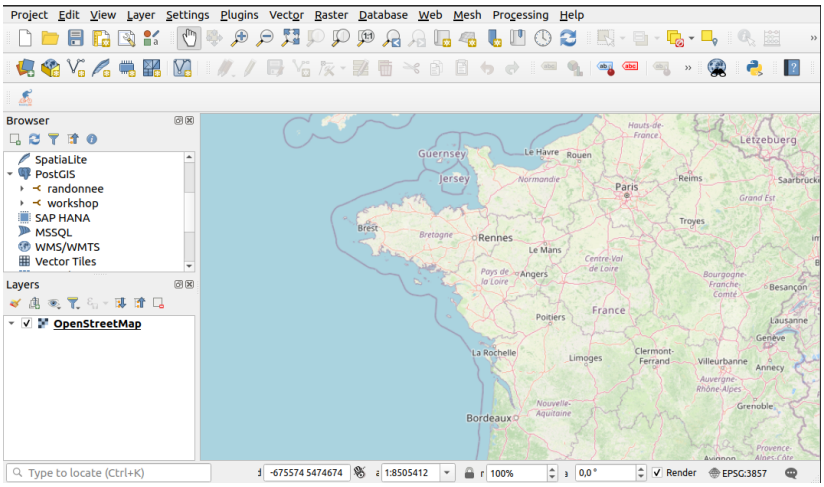

Figure 2.2: OpenStreetMap layer

```
SELECT date, trip FROM trips_mdb;
```

Now click on 'Execute Query'. Let's see the trip moving while we are visualizing it. To do so, click on View → Panels → Temporal Controller. You will see a window like this: Now this is your time to play with the data and visualize it as you want.

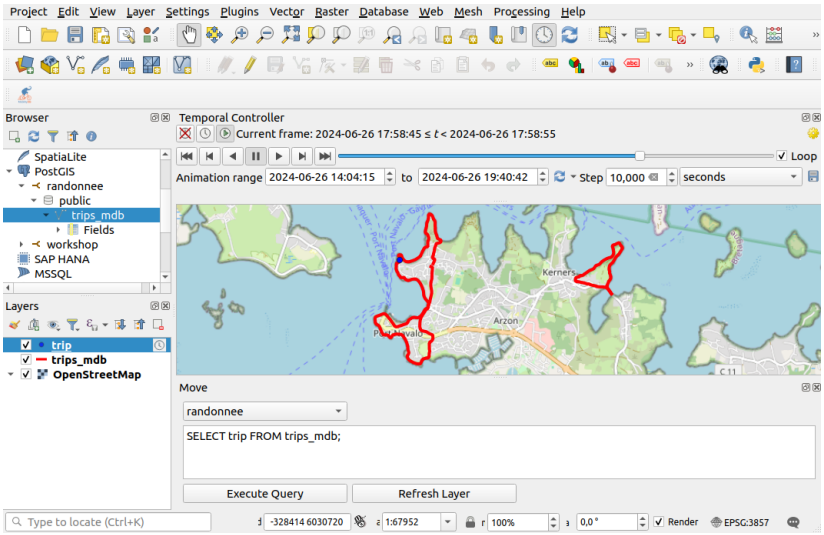

Figure 2.3: Full temporal visualization

You can put the time in the temporal controller and see the trip moving. You can also change the color of the trip, the size of the points, etc...

Figure 2.4: Visualizing the trip